PATTERN MATCHING SUITE

Comparative Analysis of String Searching Algorithms

Naive | KMP | Rabin-Karp | Boyer-Moore

Semester Project | Spring 2026

TABLE OF CONTENTS

01 Problem Overview & Motivation

02 Four Algorithms — Core Ideas

03 Algorithm Deep Dive (KMP, BM, RK)

04 C++ Implementation Details

05 Performance Results & Comparison

06 Recommendation System

07 Conclusion & Future Work

PROBLEM OVERVIEW

```
Given Text T and Pattern P → Find ALL positions where P occurs in T
```

Real-World Applications:

Text Editors
Ctrl+F search

Bioinformatics
DNA analysis

Cybersecurity
IDS signatures

Search Engines
Keyword index

Compilers
Lexical analysis

Databases
Query matching

FOUR ALGORITHMS AT A GLANCE

NAIVE

Brute Force

Worst Time:

$O(nm)$

Space:

$O(1)$

Check every position.
Simple but slow on repetitive text.

KMP

Knuth-Morris-Pratt

Worst Time:

$O(n+m)$

Space:

$O(m)$

Precompute LPS table.
Guaranteed linear time always.

RABIN-KARP

Rolling Hash

Worst Time:

$O(n+m)$ avg

Space:

$O(1)$

Hash the pattern window.
Excellent for multiple patterns.

BOYER-MOORE

Bad Character

Worst Time:

$O(n/m)$ best

Space:

$O(\sigma)$

Scan right-to-left.
Fastest in practice for long patterns.

ALGORITHM 1 — NAIVE (BRUTE FORCE)

A
1

Algorithm NAIVE(Text[ ], Pat[ ], n, m)

{ Checks every alignment of P against T one-by-one }

Step-1 [Initialize outer loop index]

Set i = 0

Repeat steps 2-3 for i = 0 to n-m, 1

Step-2 [Compare pattern at position i]

Set j = 0

Repeat while j < m and Text[i+j] = Pat[j]

Set j = j + 1

Step-3 [Check for full match]

If j = m then

Write("Pattern found at index", i)

Step-4 [Finish]

Exit

Key Properties

Time Complexity:

Best: O(n)

Worst: O(nm)

Space: O(1)

Preprocessing: None

Best For:

Short patterns

Small alphabets

Worst Case On:

Repetitive text

e.g. Text = "AAAA..."

ALGORITHM 2 — KMP (KNUTH-MORRIS-PRATT)

A
2

Algorithm KMP(Text[ ], Pat[ ], n, m)

{ Uses LPS table to skip redundant comparisons }

Step-1 [Build LPS (failure function) table]

Call BUILD_LPS(Pat, m) → lps[ ]

Step-2 [Initialize indices]

Set i = 0 { index in Text }
Set j = 0 { index in Pat }

Step-3 [Traverse text]

```
Repeat while i < n
        If Text[i] = Pat[j] then
                Increment i and j
        If j = m then
                Write("Match at", i-j); set j = lps[j-1]
        Else If i < n and Text[i] != Pat[j]
                If j != 0 then j = lps[j-1]
                Else i = i + 1
```

Step-4 [Finish]

Exit

Key Properties

Time Complexity:
All cases: O(n+m)

Space: O(m)
(for LPS array)

Preprocessing:
LPS table — O(m)

Best For:
DNA / repetitive text

Advantage:
No worst-case degradation
Never re-examines chars

ALGORITHM 3 — RABIN-KARP (ROLLING HASH)

A
3

Algorithm RABIN_KARP(Text[ ], Pat[ ], n, m, q)

{ Uses rolling hash to find pattern matches }

Step-1 [Compute initial hash values]

Set p_hash = hash(Pat[0..m-1])
Set t_hash = hash(Text[0..m-1])
Set h = (d^(m-1)) mod q

Step-2 [Slide window over text]

Repeat for i = 0 to n-m, 1

Step-3 [Check hash match]

If p_hash = t_hash then
Verify char-by-char: Text[i..i+m-1] = Pat
If equal: Write("Match at index", i)

Step-4 [Compute rolling hash for next window]

t_hash = (d(t_hash - Text[i]*h) + Text[i+m]) mod q*
If t_hash < 0 then t_hash = t_hash + q

Step-5 [Finish]

Exit

Key Properties

Time Complexity:
Avg: O(n+m)
Worst: O(nm)

Space: O(1)

Preprocessing:
Hash computation O(m)

Best For:
Multi-pattern search

Key Idea:
Rolling hash avoids recomputing each window

ALGORITHM 4 — BOYER-MOORE (BAD CHAR)

A
4

Algorithm BOYER_MOORE(Text[ ], Pat[ ], n, m)

{ Scans pattern RIGHT-TO-LEFT; skips using heuristics }

Step-1 [Preprocess: build heuristic tables]

```
Call BAD_CHAR(Pat, m)   → bc[ ]
Call GOOD_SUFFIX(Pat, m) → gs[ ]
```

Step-2 [Initialize alignment]

```
Set i = 0  { starting position of window in Text }
```

Step-3 [Slide pattern right-to-left]

```
Repeat while i <= n-m
        Set j = m - 1
        Repeat while j >= 0 and Pat[j] = Text[i+j]
                j = j - 1
        If j < 0: Write("Match at", i)
```

Step-4 [Compute skip using max heuristic]

```
shift = max(gs[j], j - bc[Text[i+j]])
Set i = i + shift
```

Step-5 [Finish]

Key Properties

Time Complexity:
- Best: O(n/m)
- Worst: O(nm)

Space: O(m + sigma)

Preprocessing:
- Bad-char + Good-suffix

Best For:
- Long patterns
- Natural language text

Key Insight:
- Skips large sections
- Sub-linear in practice

KMP — HOW THE LPS TABLE WORKS

LPS = Longest Proper Prefix that is also Suffix

Index	0	1	2	3	4	5	6	7	8
Char	A	B	A	B	C	A	B	A	B
LPS	0	0	1	2	0	1	2	3	4

When mismatch at j, jump to lps[j-1]
instead of restarting from 0.
This avoids redundant comparisons!

Comparisons Saved

Naive **~4900 cmp**

KMP **~510 cmp**
On repetitive text (500 chars)
Pattern: ABABCABABD

KMP Guarantee:
O(n + m) for ALL inputs
No worst-case degradation

BOYER-MOORE — BAD CHARACTER HEURISTIC

Key Insight: Scan the pattern RIGHT-TO-LEFT. Use mismatches to make LARGE skips.

Step 1: Compare right-to-left

mismatch at T[3]=B, P[3]=B → match, continue

Step 2: T[2]=C not in pattern → skip!

Shift by 3 (past all of pattern)

Step 3: New window aligned

Match found at position 7

Best Case: O(n/m) — Sub-linear! Examines only 1 in every m characters on average.

C++ IMPLEMENTATION

```cpp
struct SearchResult {
  string  algorithmName;
  vector<int> occurrences;
  long long   comparisons;
  double      timeMs;
  size_t      memoryBytes;
};

// KMP — O(n+m) guaranteed
SearchResult kmpSearch(
  const string& text,
  const string& pattern) {
  // ... LPS build + search
}
```

4 Algorithms
Naive, KMP, Rabin-Karp, Boyer-Moore

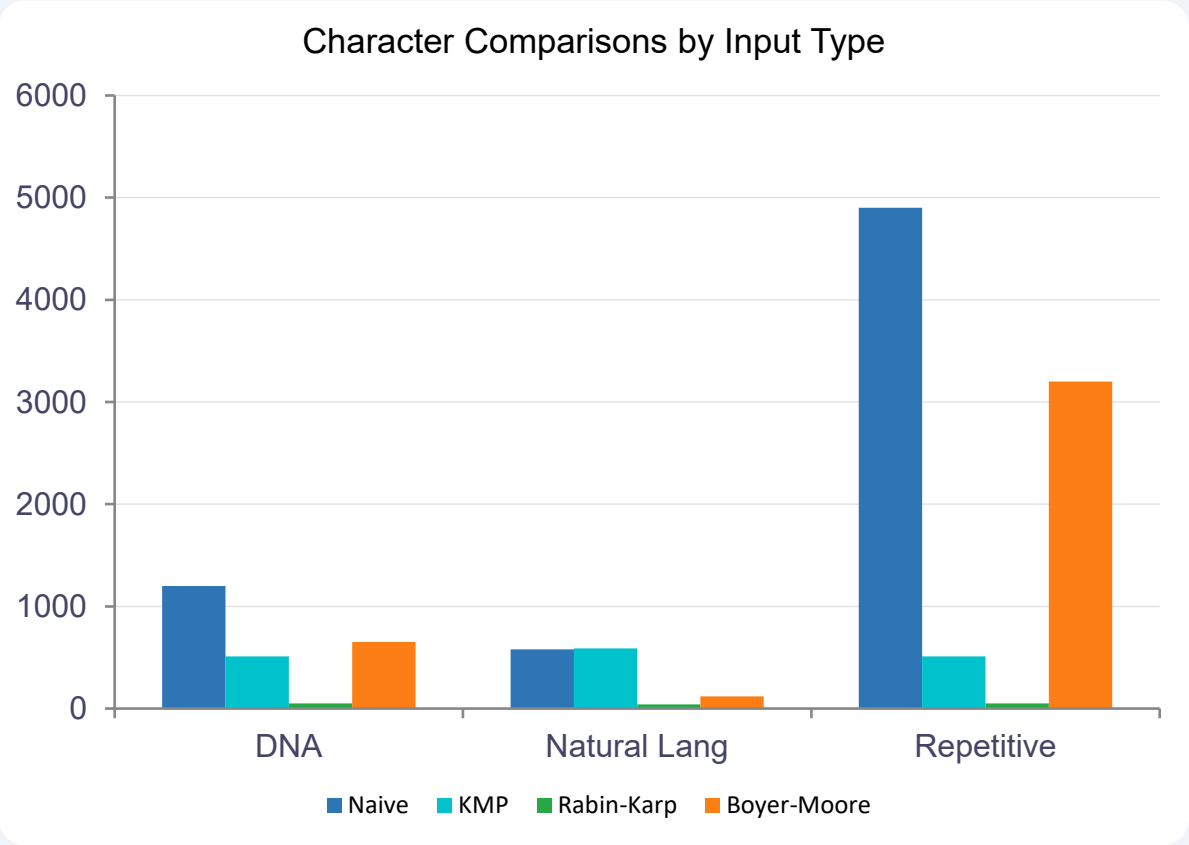
Metrics
Time (ms), comparisons, memory

Visual Map
ASCII occurrence map of matches

Recommender
Smart algorithm selection

Multi-pattern
Search multiple patterns at once

PERFORMANCE RESULTS



KMP Winner

Repetitive text

510 cmp

BM Winner

Natural language

120 cmp

RK Lowest

All scenarios

~50 cmp

ALGORITHM COMPARISON TABLE

Property	Naive	KMP	Rabin-Karp	Boyer-Moore
Best Time	O(n)	O(n+m)	O(n+m)	O(n/m) ⚡
Worst Time	O(nm) ⚠️	O(n+m) ✓	O(nm)	O(nm)
Space	O(1) ✓	O(m)	O(1) ✓	O(σ)
Preprocessing	None	LPS table	Hash val	Bad-char tbl
Best For	Short patterns	DNA/repeat	Multi-ptn	Long patterns
Worst Case On	Repetitive	None	Many hash	Small alphabet

RECOMMENDATION SYSTEM

The program analyzes your text and automatically suggests the best algorithm

IF: Pattern length ≤ 5

→ Naive or KMP

Preprocessing overhead not worth it for tiny patterns

IF: Pattern length ≥ 20 (non-repetitive text)

→ Boyer-Moore

Large bad-character skips make it sub-linear in practice

IF: DNA or highly repetitive text

→ KMP

Small alphabet kills BM skips. KMP stays O(n+m) always

IF: Natural language average-length pattern

→ KMP or BM

Both work well. KMP is safer; BM often faster in practice

HOW TO COMPILE & RUN

```
$ g++ -O2 -std=c++17 -o pattern_matching pattern_matching_suite.cpp
```

```
$ ./pattern_matching                    OR    ./pattern_matching myfile.txt
```

Interactive Menu Flow:

1

Load text

DNA / Natural / Repetitive / Custom / File

2

Enter pattern

Type the string to search for

3

Choose mode

Single algo / Compare all / Multi-pattern / Recommend

4

View results

Table + occurrence map + recommendation

```
Occurrences: 7  |  Comparisons: 510  |  Time: 0.0112 ms
[##...#.##.....#..#....#..#.##.#....#.#.....##..#..#.##..]
```

CONCLUSION

- Implemented all 4 algorithms in pure C++17
- Built visual occurrence map for intuitive output
- Measured time, comparisons, and memory precisely
- Recommendation engine selects optimal algorithm
- Tested on DNA, natural language, and repetitive text
- Learned: no single algorithm is universally best

Thank You | Questions Welcome!